PRACTICAL 7. MongoDB Basics

PRACTICAL NO.7(A)

Aim:- Install and start MongoDB.

Requirements:-

- Windows PC
- Command Prompt (CMD)
- MongoDB Server 4.4
- MongoSH

1. Install MongoDB Server

Step 1: Download MongoDB

Download the **MongoDB Community Server 4.4** installer for Windows.

Run the downloaded **.msi** file.

Step 2: Install MongoDB

1. Open the MongoDB .msi installer.
2. Click **Next**.
3. Accept the license agreement and click **Next**.
4. Select **Complete** installation.
5. Keep the default installation location:

C:\Program Files\MongoDB\Server\4.4\

6. Click **Next** and then **Install**.
7. After installation is completed, click **Finish**.

The **MongoDB executable files** will be available in:

C:\Program Files\MongoDB\Server\4.4\bin

The main server program is:

mongod.exe

2. Install MongoSH

MongoSH is the MongoDB command-line shell used to connect to the MongoDB server.

Step 1: Download MongoSH

Download the MongoSH Windows ZIP package.
After downloading, extract it to:
D:\Fullstack

The **MongoSH** folder should be:
D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\

The executable files will be inside:
D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin

The main shell program is:
mongosh.exe

3. Create MongoDB Data Directory

Open **Command Prompt as Administrator** and run:

mkdir C:\data
mkdir C:\data\db

MongoDB will use this directory to store database files.

4. Set MongoDB Server Path

In CMD, run:
setx PATH "%PATH%;C:\Program Files\MongoDB\Server\4.4\bin"

5. Set MongoSH Path

Run:

setx PATH

"%PATH%;D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin"

Close CMD and open a **new Command Prompt**.

6. Check MongoDB Installation

Run:

mongod --version

If MongoDB is installed correctly, the MongoDB server version will be displayed.

7. Check MongoSH Installation

Run:

mongosh --version

If MongoSH is installed correctly, the MongoSH version will be displayed.

8. Start MongoDB Server

Run:

mongod --dbpath C:\data\db

MongoDB will start and listen on the default port:

27017

Keep this CMD window open.

9. Start MongoSH

Open a **second Command Prompt** window and type:

mongosh

MongoSH will connect to the MongoDB server.

You should see a connection similar to:
mongodb://127.0.0.1:27017/

This means MongoDB is running successfully.

10. Test MongoDB

Inside MongoSH, run:
show dbs

Create a database:

use college

Create a collection and insert data:

→ db.students.insertOne({name:"Rahul", age:20})

Display the data:

→ db.students.find()

11. Check the PATH

To check MongoDB Server path:

where mongod

Expected:

C:\Program Files\MongoDB\Server\4.4\bin\mongod.exe

To check MongoSH path:

where mongosh

Expected:

D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin\mongosh.exe

12. Stop MongoDB

To stop the MongoDB server, go to the CMD window where **mongod** is running and press:

Ctrl + C

Environment Variable Paths

Add these two paths to the Windows **Path** environment variable:

→ C:\Program Files\MongoDB\Server\4.4\bin

→ D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin

```
C:\Windows\System32>mongod --version
db version v4.4.3
Build Info: {
    "version": "4.4.3",
    "gitVersion": "913d6b62acfbb344dde1b116f4161360acd8fd13",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "windows",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}

C:\Windows\System32>mongosh --version
2.12.0

C:\Windows\System32>net start MongoDB
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.
```

```
C:\Windows\System32>net start MongoDB
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.


C:\Windows\System32>mongosh
Current Mongosh Log ID: 6abdcc75e69ddfbd677d76fc
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:          4.4.3
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-01T07:42:59.284+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test>
```

PRACTICAL NO.7(B)

Aim:- Create a database and collection.

Create/Select a database called collegeDB1:

→ use collegeDB1

Output:-

```
test> use collegeDB1
switched to db collegeDB1
collegeDB1>
```

Create Collection:-

→ db.createCollection("students")

Output:-

```
collegeDB1> db.createCollection("student")
{ ok: 1 }
```

Show all Collections:-

→ show collections

Output:-

```
collegeDB1> show collections
student
students
```

PRACTICAL NO.7(C)

Aim:- Insert records in MongoDB.

Insert One value:-

→db.student.insertOne({

Name:"Rahul",

Age:20,

```
        Course:"BCA",
        City:"Mumbai"
})
```

Output:-

```
collegeDB1> db.student.insertOne({ Name:"Rahul",Age:20,Course:"BCA",City:"Mumbai"})
{
  acknowledged: true,
  insertedId: ObjectId('6abdd464e69ddfbd677d76fd')
}
```

Insert Many values:-

```
→ db.student.insertMany([
        {
                Name:"Priya",
                Age:21,
                Course:"BCA",
                City:"Pune"
        },
        {
                Name:"Amit",
                Age:22,
                Course:"BSc",
                City:"Nashik"
        },
        {
                Name:"Sneha",
                Age:20,
                Course:"BCA",
                City:"Nagpur"
        }
])
```

Output:-

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abdd684e69ddfbd677d76fe'),
    '1': ObjectId('6abdd684e69ddfbd677d76ff'),
    '2': ObjectId('6abdd684e69ddfbd677d7700')
  }
}
```

PRACTICAL NO.7(D)

Aim:- View records from the collection.

View Record from collection:-

→ db.student.find()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76ff'),
    Name: 'Amit',
    Age: 22,
    Course: 'BSc',
    City: 'Nashik'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

View a particular record:-

→db.student.find({Name:"Rahul"})

Output:-

```
collegeDB1> db.student.find({Name:"Rahul"})
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  }
]
```

View all record in cleaner format:-

→ db.student.find().pretty()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76ff'),
    Name: 'Amit',
    Age: 22,
    Course: 'BSc',
    City: 'Nashik'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

Update a record:-

→db.student.updateOne({Name:"Rahul"},{$set:{City:"Thane"}})

Output:-

```
collegeDB1> db.student.updateOne({Name:"Rahul"},{$set:{City:"Thane"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Check the updated record:-

→ db.student.find({Name:"Rahul"})

Output:-

```
collegeDB1> db.student.find({Name:"Rahul"})
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Thane'
  }
]
```

PRACTICAL NO.7(E)

Aim:- Update and delete records.

Delete a specific record:-

→ db.student.deleteOne({Name:"Amit"})

Output:-

```
collegeDB1> db.student.deleteOne({Name:"Amit"})
{ acknowledged: true, deletedCount: 1 }
```

Check if record is deleted:-

→ db.student.find()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Thane'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

PRACTICAL NO.7(F)

Aim:- Connect MongoDB with Node.js.

STEP 1: Open your project folder in VS Code

1. Open Visual Studio Code.
2. Click:

   File → Open Folder
3. Select this folder:

   C:\Users\user\mongodb-node-practical13
4. Click "Select Folder".

5. You should see these files/folders in the VS Code Explorer:
   - node_modules
   - package-lock.json
   - package.json

STEP 2: Create server.js

1. In VS Code, look at the Explorer panel on the left.

2. Right-click the project folder:
   Mongodb-node-practical13

3. **Select:**
   New File

4. **Enter:**
   server.js

5. Press Enter.

6. Open server.js.

STEP 3: Check your project structure

Your project should now look like:

```
mongodb-node-practical13
│
├── node_modules
│
├── package-lock.json
│
├── package.json
│
└── server.js
```

STEP 4: Add MongoDB connection code

Copy the following code into server.js:

```
const { MongoClient } = require("mongodb");

const uri = "mongodb://127.0.0.1:27017";

const client = new MongoClient(uri);

async function connectDB() {
   try {
      await client.connect();

      console.log("MongoDB connected successfully!");

      const db = client.db("practical13");

      console.log("Database selected:", db.databaseName);
   } catch (error) {
      console.error("MongoDB connection failed:", error);
   }
}

connectDB();
```

STEP 5: Make sure MongoDB Server is running

MongoDB must be running before you execute server.js.

If MongoDB was installed as a Windows service, open Command Prompt and check:

net start MongoDB

If it says that the MongoDB service is already started, that is fine.

If MongoDB is not installed/running as a Windows service, start MongoDB using the MongoDB installation method you used.

STEP 6: Open the VS Code terminal

In VS Code:
Terminal → New Terminal
The terminal should open at your project folder.

It should look similar to:
PS C:\Users\user\mongodb-node-practical13>

STEP 7: Run server.js

In the VS Code terminal, type: **node server.js**

Press Enter.

STEP 8: Check the output

If everything is working correctly, you should see:

MongoDB connected successfully!
Database selected: practical13

This means Node.js successfully connected to your local MongoDB server.